CAHIER DES CHARGES

Software Requirements Specification

Camp-Compass
University Campus Mapping & Timetable Management System

Document Reference	CDC-CampCompass-2026-v1.0
Version	1.0
Date	March 2026
Status	Draft
Classification	Confidential
Target Audience	Universities, Training Centres, Large Schools

Table of Contents

Table of Contents 2
1. Introduction 4
1.1 Project Overview 4
1.2 Project Objectives 4
1.3 Scope 4
1.4 Definitions and Acronyms 4
2. Stakeholders and System Actors 6
2.1 Stakeholders 6
2.2 System Actors 6
3. Functional Requirements 7
3.1 Authentication and User Management 7
3.1.1 Login 7
3.1.2 Access Control 7
3.2 Campus Mapping Module 7
3.2.1 Outdoor Campus Maps 7
3.2.2 Indoor Floor Plans 8
3.2.3 Multi-Campus Support 8
3.3 Timetable Management Module 8
3.3.1 Timetable Generation 8
3.3.2 Timetable Viewing — Students 8
3.3.3 Timetable Viewing — Staff 9
3.3.4 Lecturer Availability Declaration 9
3.3.5 Hall Management 9
3.3.6 Hall Search 9
3.3.7 Map Integration from Timetable 9
3.4 Notification Module 10
3.5 Registration and Onboarding Module 10
4. Non-Functional Requirements 11
4.1 Availability and Offline Mode 11
4.2 Cross-Platform Compatibility 11
4.3 Performance 11
4.4 Security 11
4.5 Maintainability and Scalability 11
4.6 Accessibility 11
5. Technical Architecture 12
5.1 Technology Stack 12

5.2 System Architecture Overview 12
5.3 Data Model — Key Entities 12
6. Key Use Cases 14
6.1 UC-01: Student Views Campus Map 14
6.2 UC-02: Student Views Timetable 14
6.3 UC-03: Lecturer Declares / Updates Availability 14
6.4 UC-04: Admin Generates and Adjusts Timetable 14
7. Constraints and Assumptions 16
7.1 Technical Constraints 16
7.2 Business Constraints 16
7.3 Assumptions 16
8. Project Deliverables and Milestones 17
8.1 Deliverables 17
8.2 Suggested Development Milestones 17
9. Acceptance Criteria 18
10. Document Revision History 18

1. Introduction

1.1 Project Overview

This document constitutes the formal Cahier des Charges (Software Requirements Specification) for the University Campus Mapping and Timetable Management System, named Camp-Compass (UCMTMS). It describes the functional and non-functional requirements, system architecture, actors, constraints, and technical stack for the development of the application.

The Application name is **Camp-Compass**.

1.2 Project Objectives

The UCMTMS aims to provide an integrated digital platform that fulfills two primary functions:

- Interactive Campus Mapping: Provide accurate, interactive maps of university campuses including outdoor navigation via Google Maps integration and indoor navigation via interactive floor plan drawings for each building.
- Automated Timetable Generation and Viewing: Automate the creation of weekly academic timetables, taking into account courses, lecturer availability, hall capacities, and scheduling constraints, while providing role-specific views for students and staff.

1.3 Scope

The application is designed for deployment in:

- Universities and higher education institutions
- Professional training centers
- Large secondary schools

The system shall support multi-campus institutions, including campuses located in different cities and regions within the same country. The application shall be accessible via web browsers and mobile devices (iOS and Android), and shall function in both online and offline modes.

1.4 Definitions and Acronyms

Term / Acronym	Definition
UCMTMS	Camp-Compass — University Campus Mapping and Timetable Management System
SRS	Software Requirements Specification
Admin	Administrator actor with full system access
Register	Registrar actor responsible for managing student/staff records
PWA	Progressive Web Application — enables offline use and mobile install
Next.js	React-based full-stack web framework used for development
Google Maps API	Google mapping service used for outdoor campus navigation

Floor Plan	Interactive image-based indoor map of a building floor
Hall	A teaching room, lecture theatre, or laboratory in the institution
Timetable	A weekly schedule of courses mapped to halls, time slots, and lecturers
Tuition Gate	Access restriction applied to students with unpaid first tuition instalment

2. Stakeholders and System Actors

2.1 Stakeholders

The primary institutional clients of the UCMTMS are:

- Universities and higher education institutions seeking to digitise campus navigation and academic scheduling.
- Professional training centres that need structured timetabling and campus orientation tools.
- Large secondary schools requiring hall management and class scheduling.

2.2 System Actors

The system defines four main actors, each with distinct roles and access levels:

Actor	Access Level	Description
Student	Restricted / Standard	Enrolled learner. Limited to mapping features until first tuition instalment is confirmed. Once fees are cleared, full timetable access is granted for their department and level only.
Staff	Standard	Institution employee. Can view campus maps, consult their personal weekly schedule, and — if a lecturer — declare or update their availability for the following week.
Admin	Elevated	System administrator. Manages timetables, receives lecturer availability notifications, manually adjusts schedules mid-week, and sends notifications to affected students.
Register (Registrar)	Elevated	Responsible for registering students and staff members into the system, generating auto-assigned login credentials (email and password), and managing enrolment records.

3. Functional Requirements

3.1 Authentication and User Management

3.1.1 Login

- All users (Students, Staff, Admin, Registrar) shall authenticate using an institutional email address and an auto-generated password.
- Credentials are generated by the Registrar upon entry of a new member into the institution's system.
- Users shall be able to change their auto-generated password on first login.

3.1.2 Access Control

- The system shall enforce role-based access control (RBAC) for all features.
- Students with an unpaid first tuition instalment shall only have access to the campus mapping module. All other features (timetable, notifications) shall be locked until payment is confirmed.
- Staff members shall access their personal schedule and the mapping module.
- Admin and Registrar actors shall have access to their respective management dashboards.

3.1.3 Account Profile and Detail Correction Request

- All users (Students, Staff, Admin, and Registrar) shall be able to view their own account details within a dedicated Profile section of the application. The profile shall display information including full name, institutional email, role, department or faculty, and academic level where applicable.
- Users shall not be able to edit their account details directly. If a user identifies an error in their profile — such as a misspelling of their name or an incorrect department or faculty attribution — they may submit a formal Detail Correction Request through the application.
- When submitting a Detail Correction Request, the user shall be required to provide the following:

A clear description of the incorrect information and the correct value being requested. Supporting evidence in the form of one or more of the following: an official identity or academic document (e.g., national ID, student card, or enrolment certificate), a payment receipt, or an employment contract. Evidence shall be uploaded as an image or PDF copy directly within the request form.

- Submitted correction requests shall be routed to an Admin for review and validation. The Admin shall be able to approve or reject the request from the administration dashboard, optionally providing a reason for rejection.
- Upon Admin approval, the relevant account detail shall be updated in the system automatically. The user shall receive an in-app notification confirming whether their request was approved or rejected.
- A user may only have one pending correction request at a time per field. Uploaded evidence documents shall be stored securely and accessible only to Admin users.

3.2 Campus Mapping Module

3.2.1 Outdoor Campus Maps

- The system shall display interactive maps of all campuses of the institution using the Google Maps API.
- Each campus shall be identifiable and selectable, even if located in different cities or regions.
- The map shall display buildings, entrances, parking areas, and key points of interest with labelled markers.
- Users shall be able to search for specific buildings or locations within a campus.

3.2.2 Indoor Floor Plans

- Each building shall have one or more interactive floor plan drawings, one per floor.
- Floor plans shall be displayed as interactive image-based drawings (SVG or annotated raster images).
- Users shall be able to navigate between floors within a building.
- Rooms and halls shall be labelled and clickable, showing details such as room name, capacity, and type.
- The system shall support linking a timetable entry directly to its hall's location on the floor plan (see section 3.3.6).

3.2.3 Multi-Campus Support

- The system shall support institutions with multiple campuses across different cities and regions.
- A campus selector shall allow users to switch between campuses.
- All mapping data shall be stored and retrieved on a per-campus basis.

3.3 Timetable Management Module

3.3.1 Timetable Generation

- The Admin shall be able to trigger weekly timetable generation.
- The system shall automatically generate timetables taking into account the following constraints:
  - Courses assigned to each department and academic level.
  - Lecturer availability for the week (as declared by lecturers).
  - Hall capacity relative to the number of students in the class.
  - Hall type (e.g., laboratory, lecture theatre, seminar room) matching the course requirements.
  - Absence of scheduling conflicts: a hall cannot be assigned to two courses at the same time slot.
  - Absence of lecturer conflicts: a lecturer cannot teach two courses simultaneously.

3.3.2 Timetable Viewing — Students

- Students shall view timetables filtered exclusively to their enrolled department and academic level.
- No cross-departmental or cross-level timetable data shall be visible to students.
- The timetable shall display: day, time slot, course name, lecturer name, and hall designation.

3.3.3 Timetable Viewing — Staff

- Staff members who are lecturers shall view their personal weekly teaching schedule.
- Non-teaching staff shall have access to general campus timetable information relevant to their role.

3.3.4 Lecturer Availability Declaration

- Lecturers shall declare their availability for the following week before a configurable weekly deadline.
- Lecturers shall also be permitted to edit their availability mid-week (e.g., due to unforeseen circumstances).
- Any mid-week availability change shall trigger an automatic notification to the Admin.
- The Admin shall manually update the affected timetable entries accordingly.
- Students of the impacted department and level shall be notified of the timetable change.

3.3.5 Hall Management

- The Admin shall manage the list of halls, including their name, campus, building, floor, capacity, and type.
- Halls shall be marked as available or unavailable for scheduling.
- The system shall prevent double-booking of halls.

3.3.6 Hall Search

- The system shall provide a dedicated Hall Search interface accessible to Admin users and, in read-only mode, to authorised Staff members.
- Users shall be able to search for halls using one or more of the following criteria:

Hall Code: unique identifier assigned to each hall (e.g., “A-101”).
Building Code: identifier of the building in which the hall is located (e.g., “BLD-03”).
Hall Name: full or partial name of the hall.
Hall Type: lecture theatre, laboratory, seminar room, etc.
Campus: filter halls by campus.
Capacity: minimum seating capacity required.
Availability Status: available or unavailable for scheduling.

- Search criteria may be combined to perform multi-criteria filtering. Results shall update in real time as criteria are entered.
- Each result in the hall search list shall be clickable, directing the user to the hall’s details page and its location on the relevant floor plan.
- The backend shall expose a dedicated REST endpoint (e.g., GET /api/halls/search) accepting query parameters for all supported search criteria. The Python backend layer shall implement the filtering logic using parameterised database queries to ensure security and performance.

3.3.7 Map Integration from Timetable

- From any timetable entry, a student or staff member shall be able to click on the hall name to view its location on the campus map.
- The system shall navigate to the correct campus, building, and floor plan, highlighting the relevant hall.

3.4 Notification Module

- The system shall support in-app and push notifications.
- Notification triggers include:
  - Timetable changes affecting a student's department and level.
  - Lecturer availability updates received by the Admin.
  - System announcements from the Admin.
- Approval or rejection of a user's Detail Correction Request (see section 3.1.3).
- Students shall only receive notifications relevant to their department and level.

3.5 Registration and Onboarding Module

- The Registrar shall register new students and staff into the system.
- Upon registration, the system shall auto-generate an institutional email address and a temporary password.
- Registration details shall include: full name, role, department (for students), faculty/department (for staff), and academic level (for students).
- A student's access tier (restricted vs. standard) shall be linked to their tuition fee payment status.

4. Non-Functional Requirements

4.1 Availability and Offline Mode

- The application shall function in both online and offline modes using Progressive Web Application (PWA) techniques and local data caching.
- Campus maps and the most recently fetched timetable shall be accessible offline.
- Data synchronisation shall occur automatically when connectivity is restored.

4.2 Cross-Platform Compatibility

- The application shall be accessible via all major web browsers (Chrome, Firefox, Edge, Safari).
- The application shall be installable on Android and iOS devices as a PWA.
- The user interface shall be fully responsive and optimised for mobile screens.

4.3 Performance

- Map tiles and floor plan images shall load within 3 seconds on a standard 4G mobile connection.
- Timetable generation shall complete within 30 seconds for a standard institution configuration.
- The application shall support a minimum of 5,000 concurrent users without performance degradation.

4.4 Security

- All data transmissions shall be encrypted using HTTPS/TLS.
- Passwords shall be hashed using a strong one-way hashing algorithm (e.g., bcrypt).
- Role-based access control shall be enforced server-side.
- Session tokens shall expire after a configurable inactivity period.

4.5 Maintainability and Scalability

- The application shall be designed to support multiple institution clients (multi-tenancy architecture).
- The codebase shall follow modular design principles to facilitate future feature additions.
- The system shall support the addition of new campuses and buildings without architectural changes.

4.6 Accessibility

- The interface shall comply with WCAG 2.1 Level AA accessibility standards.
- Text labels and alt-text shall be provided for all map icons and floor plan elements.

5. Technical Architecture

5.1 Technology Stack

Layer	Technology
Frontend Framework	React / Next.js (App Router)
Styling	Tailwind CSS
Maps (Outdoor)	Google Maps JavaScript API / Google Maps Platform
Maps (Indoor)	Custom interactive SVG / annotated image floor plans
Offline Support	Progressive Web App (PWA) with Service Workers and Cache API
Backend / API	Python (FastAPI or Django REST Framework) for business logic and REST API; Next.js API Routes for frontend-proxied calls and server-side rendering
Authentication	NextAuth.js (Credentials provider)
Database	PostgreSQL (recommended) with Prisma ORM
Notifications	Web Push API / Firebase Cloud Messaging (FCM)
Deployment	Vercel (recommended) or self-hosted Node.js server
Version Control	Git / GitHub or GitLab

5.2 System Architecture Overview

The system follows a client-server architecture with the following key components:

- Client Layer: Next.js application rendering UI for web and PWA-enabled mobile devices. Service Workers manage offline caching of timetables and map data.
- API Layer: A Python backend (FastAPI or Django REST Framework) exposes RESTful endpoints for authentication, timetable management, hall management, hall search, map data, notifications, and user management. Next.js API routes handle frontend-proxied calls and server-side rendering concerns.
- Business Logic Layer: Server-side modules handle timetable constraint solving, availability processing, notification dispatching, and access control enforcement.
- Data Layer: PostgreSQL relational database stores all application data. Prisma ORM is used for type-safe database queries. Multi-tenancy is implemented via institution-scoped data models.
- External Services: Google Maps Platform for outdoor cartography. Firebase Cloud Messaging or Web Push API for push notifications.

5.3 Data Model — Key Entities

The principal data entities of the system include:

- Institution: top-level tenant entity containing campuses, departments, and configuration.
- Campus: belongs to an Institution, has a geographic location, and contains Buildings.
- Building: belongs to a Campus, has a name and address, and contains Floors.

- Floor: belongs to a Building, has a floor number and an associated floor plan image/SVG.
- Hall: belongs to a Floor, has a name, capacity, type, and availability status.
- Department: belongs to an Institution, groups Courses and Students by academic field.
- Level: represents an academic year or semester within a Department.
- Course: belongs to a Department and Level, has a name and weekly hour allocation.
- User: polymorphic entity covering Student, Staff, Admin, and Registrar actors. Stores institutional email, hashed password, role, and associated department/level.
- Timetable Entry: maps a Course, Hall, Lecturer, time slot, and week to a scheduled class.
- Availability: records a lecturer's declared availability by day and time slot for a given week.
- Notification: stores notification records with target actor type, department, level, and message.

6. Key Use Cases

6.1 UC-01: Student Views Campus Map

Use Case ID	UC-01
Actor	Student
Precondition	Student is logged in (any access tier)
Main Flow	1. Student opens the Maps module. 2. System displays the campus selector. 3. Student selects a campus. 4. Google Maps view loads with building markers. 5. Student clicks on a building marker. 6. System displays building details and floor selector. 7. Student selects a floor to view the interactive floor plan.
Alternate Flow	If offline: cached map data is loaded instead of live tiles.
Postcondition	Student views the selected floor plan.

6.2 UC-02: Student Views Timetable

Use Case ID	UC-02
Actor	Student
Precondition	Student is logged in and first tuition instalment is confirmed.
Main Flow	1. Student opens the Timetable module. 2. System automatically filters timetable by student's department and level. 3. Weekly timetable is displayed. 4. Student clicks on a timetable entry. 5. System shows course details and hall location. 6. Student clicks on hall name to view it on the floor plan map.
Exception	If tuition is unpaid: timetable module is locked; system displays payment reminder.
Postcondition	Student views their filtered weekly timetable and can navigate to the hall on the map.

6.3 UC-03: Lecturer Declares / Updates Availability

Use Case ID	UC-03
Actor	Staff (Lecturer)
Precondition	Lecturer is logged in and has the Lecturer sub-role.
Main Flow	1. Lecturer opens the Availability module. 2. System displays a weekly availability grid. 3. Lecturer selects available time slots for the following week. 4. Lecturer submits their availability. 5. System saves availability and notifies Admin if this is a mid-week update.
Postcondition	Availability is recorded. If mid-week, Admin is notified.

6.4 UC-04: Admin Generates and Adjusts Timetable

Use Case ID	UC-04
Actor	Admin
Precondition	Admin is logged in. Lecturer availabilities have been submitted.
Main Flow	1. Admin opens the Timetable Management dashboard. 2. Admin triggers timetable generation for a specific week. 3. System runs the scheduling algorithm and produces a draft timetable. 4. Admin reviews and approves or manually adjusts entries. 5. Admin publishes the timetable. 6. Affected students and staff are notified.
Alternate Flow	Mid-week: Admin receives availability change notification, edits specific entries, notifies students.
Postcondition	Timetable is published and visible to authorised users.

7. Constraints and Assumptions

7.1 Technical Constraints

- The frontend must be built using React and Next.js as the primary framework. The backend must be implemented in Python using FastAPI or Django REST Framework.
- Outdoor mapping must use Google Maps Platform (API key required).
- The application must support offline use via PWA service workers.
- The system must be deployable on standard cloud infrastructure (Vercel, AWS, or equivalent).

7.2 Business Constraints

- The system must support multi-tenancy to serve multiple institutions from a single deployment.
- Student access to timetable features is gated by tuition payment status, which must be integrated with the institution's financial records or manually managed by the Registrar.
- Timetable generation must respect all scheduling constraints without manual intervention beyond final approval.

7.3 Assumptions

- Each institution provides accurate and up-to-date course, hall, and staff data upon onboarding.
- Lecturers will consistently declare their availability before the configured weekly deadline.
- The institution has access to Google Maps API credentials.
- Floor plan drawings will be provided by the institution in a digital format (SVG or high-resolution image) and uploaded by the Admin during setup.
- Tuition payment status is maintained by the Registrar or integrated with an external finance system.

8. Project Deliverables and Milestones

8.1 Deliverables

- Fully functional web application accessible via browser and installable as PWA on mobile.
- Admin dashboard for timetable management, hall management, and user administration.
- Registrar dashboard for student and staff registration and credential management.
- Interactive campus maps with Google Maps integration and indoor floor plan support.
- Automated timetable generation engine with constraint satisfaction.
- Push notification system for timetable changes.
- Comprehensive user documentation and admin guide.
- Source code repository with documentation.

8.2 Suggested Development Milestones

Phase	Scope	Key Deliverable
Phase 1	Project Setup, Architecture, DB Schema, Authentication	Working login/logout for all 4 roles
Phase 2	Campus Map Module (Google Maps + Floor Plans)	Interactive multi-campus map with floor plans
Phase 3	Hall & Course Management (Admin)	Hall and course CRUD in Admin dashboard
Phase 4	Timetable Generation Engine	Automated weekly timetable with constraint solving
Phase 5	Timetable Viewing (Student & Staff)	Role-filtered timetable views with map linking
Phase 6	Availability Module & Notifications	Lecturer availability + push notifications
Phase 7	Offline Mode (PWA), Testing, & UAT	Full offline support + user acceptance testing
Phase 8	Deployment & Documentation	Production deployment + user guides

9. Acceptance Criteria

The following criteria must be satisfied for the project to be considered complete and accepted:

- All four actor roles (Student, Staff, Admin, Registrar) can log in with auto-generated credentials.
- Students with unpaid tuition see only the map module; students with confirmed fees see their full timetable.
- Campus maps correctly display all campuses of the institution, including those in different cities.
- Floor plans for all buildings are accessible and navigable floor by floor.
- Clicking on a timetable hall entry navigates the user to the correct floor plan location.
- The timetable generation engine produces conflict-free schedules respecting all defined constraints.
- Lecturers can declare and update availability; mid-week updates notify the Admin.
- Admin can manually modify a timetable entry and affected students receive a push notification.
- The application functions correctly in offline mode for maps and timetable viewing.
- The application is fully functional on Chrome, Firefox, Safari (web) and installable on Android and iOS as a PWA.
- Performance benchmarks are met: map load under 3 seconds, timetable generation under 30 seconds.

10. Document Revision History

Version	Date	Author	Changes
1.0	March 2026	[Author Name]	Initial version

— End of Document —